

# Project 6a: DNS Security Remediation - Firewall Rule Implementation

## Complete Implementation Report

---

### Executive Summary

**Project Challenge:** Implement firewall rules to force all DNS traffic through approved infrastructure (pfSense and Pi-hole) to close the detection gap identified in Project 6, where DNS tunneling completely bypassed all security monitoring.

**Solution Attempted:** Systematically tested five different firewall and NAT-based approaches to restrict DNS traffic to approved servers only, preventing clients from bypassing security infrastructure through direct connections to unauthorized DNS servers.

**Outcome: PROJECT FAILED** - All attempted solutions proved incompatible with pfSense DNS Resolver architecture. Every enforcement mechanism that successfully blocked unauthorized DNS also broke legitimate DNS resolution, causing complete network outages. After 3+ hours of systematic testing, the DNS bypass vulnerability remains unresolved.

**Root Cause:** pfSense DNS Resolver in forwarding mode requires complex internal traffic flows between multiple interfaces (localhost, LAN, internal processes) that cannot be adequately modeled in firewall rules. Any restriction on port 53 traffic interferes with legitimate DNS forwarding operations, causing SERVFAIL responses to clients.

**Critical Finding:** The detection gap from Project 6 **cannot be closed** using pfSense firewall or NAT rules without fundamental DNS architecture redesign. Organizations using pfSense DNS Resolver in forwarding mode face an inherent limitation where client DNS enforcement through firewall rules is architecturally incompatible with service operation.

**Recommendation:** Accept DNS tunneling as an undetected attack vector in current architecture, OR pursue major DNS infrastructure redesign (estimated 2-3+ hours, high risk of extended outages). Document this as a known limitation and focus security efforts on attack vectors where detection capability exists (Wazuh host-based detection, Zeek network monitoring, pfSense perimeter logging).

---

### Version History

| Version | Date            | Author            | Changes                                                   |
|---------|-----------------|-------------------|-----------------------------------------------------------|
| 1.0     | October 1, 2025 | Prageeth Panicker | Initial implementation attempts and failure documentation |

---

# Table of Contents

1. Project Background
  2. Objectives
  3. Prerequisites
  4. Implementation Attempts
  5. Failure Analysis
  6. Alternative Approaches Considered
  7. Conclusions and Recommendations
  8. References
- 

## Project Background

### Context from Project 6

Project 6 successfully demonstrated DNS tunneling attack techniques using dnscat2, establishing an encrypted command-and-control channel between compromised target (DVWA at 192.168.30.90) and attacker infrastructure (Kali at 192.168.10.4).

**Critical finding:** The DNS tunnel completely bypassed ALL deployed security controls:

- Pi-hole DNS filtering: 0 queries logged
- pfSense firewall logs: 0 entries captured
- pfSense DNS Resolver logs: 0 tunnel queries visible
- Zeek network monitoring: 0 connections detected

**Root cause identified:** Clients directly connected to attacker-controlled DNS server (192.168.10.4:53), completely circumventing organization DNS infrastructure. Current firewall policy allows unrestricted outbound UDP/TCP port 53 to any destination.

### Project 6a Objective

Implement firewall-based enforcement to **force** all DNS traffic through approved infrastructure, eliminating the ability for clients to bypass security controls through direct connections to unauthorized DNS servers.

**Target architecture:**

Client DNS Query → Blocked unless destination is:

- 192.168.10.1 (pfSense)
- 192.168.10.106 (Pi-hole)

Expected result: DNS tunneling attempts fail, logged as blocked connections

---

## Objectives

**Primary Objective:** Implement firewall rules on pfSense that restrict all outbound DNS traffic (UDP/TCP port 53) to approved DNS servers only, preventing DNS-based attacks from bypassing security monitoring infrastructure.

### Success Criteria:

1. Clients can resolve DNS through pfSense and Pi-hole normally
2. Attempts to query unauthorized DNS servers (e.g., 8.8.8.8, attacker servers) are blocked
3. Blocked DNS attempts are logged for security analysis
4. DNS tunnel re-test fails with logged block events
5. Legitimate DNS functionality maintained for all network services

### Learning Objectives:

- Advanced pfSense firewall rule development and ordering
- Understanding firewall rule processing and state handling
- NAT redirect techniques for traffic interception
- DNS architecture constraints in enterprise firewall platforms
- Troubleshooting complex network service failures

---

## Prerequisites

### Infrastructure State

#### Current Network Configuration:

- pfSense A (MAC-MINI): 192.168.10.1 managing Management network
- Pi-hole VM: 192.168.10.106 (Ubuntu Server 22.04)
- Kali Attack VM: 192.168.10.4
- pfSense DNS Resolver: Forwarding Mode enabled, System DNS: 192.168.10.106
- DNS Resolver Network Interfaces: All (required for internal operations)

- DNS Resolver Outgoing Interfaces: LAN

### Existing Firewall Rules (Pre-Project 6a):

1. Anti-Lockout Rule
2. Allow Management Access to pfSense VM
3. Kali attack traffic documentation
4. Management access to Internal network web services
5. Default allow LAN to any rule [Problem: Permits DNS bypass]
6. Allow Management access to User-LAN network

**Known Issue:** Default allow rule permits ALL outbound traffic including DNS to any destination, enabling complete bypass of security infrastructure.

### Backup Status

**Configuration Backup Created:** pfsense-backup-before-NAT-redirect-2025-10-01.xml

- Backup location: Downloaded to local system
- Recovery capability: Verified before attempting changes

---

## Implementation Attempts

### Attempt 1: Basic Allow/Block Firewall Rules

**Approach:** Create firewall rules to explicitly allow DNS to approved servers, then block all other DNS traffic.

#### Implementation:

#### Step 1: Create Firewall Alias for Approved DNS Servers

Navigate: Firewall → Aliases  
Name: Approved\_DNS\_Servers  
Type: Host(s)  
Hosts:  
- 192.168.10.1 (pfSense)  
- 192.168.10.106 (Pi-hole)  
Description: Allowed DNS servers for Management Network

**Result:** Alias created successfully

#### Step 2: Create Allow Rule

Action: Pass  
Interface: LAN  
Protocol: TCP/UDP  
Source: LAN net (192.168.10.0/24)  
Destination: Approved\_DNS\_Servers  
Destination Port: DNS (53)  
Log: Enabled  
Description: Allow clients DNS to approved servers only

**Result:** Rule created and positioned above default allow rule

### Step 3: Create Block Rule

Action: Block  
Interface: LAN  
Protocol: TCP/UDP  
Source: LAN net (192.168.10.0/24)  
Destination: any  
Destination Port: DNS (53)  
Log: Enabled  
Description: Block DNS to unauthorized servers

**Result:** Rule created, positioned after allow rule

### Testing:

Test 1: Access pfSense web interface  
Result: SUCCESS

Test 2: nslookup google.com  
Result: FAILURE - "server failure" from 192.168.10.1

Test 3: Browse to websites  
Result: FAILURE - Unable to reach any sites

**Failure Analysis:** pfSense accessible but DNS resolution completely broken. The block rule interfered with pfSense DNS Resolver's internal forwarding operations to Pi-hole.

**Rollback:** Disabled block rule → Network access restored immediately

---

### Attempt 2: Invert Match Logic

**Approach:** Use "invert match" on destination to create more specific block rule targeting only unauthorized servers.

**Theory:** Instead of "block DNS to any", use "block DNS to NOT(Approved\_DNS\_Servers)" to avoid interfering with legitimate traffic.

### Implementation:

Modified Block Rule:

Action: Block  
Interface: LAN  
Protocol: TCP/UDP  
Source: LAN net  
Destination: Approved\_DNS\_Servers  
Destination Invert Match: CHECKED  
Destination Port: DNS (53)  
Log: Enabled  
Description: Block DNS to unauthorized servers

**Logic:** Block port 53 to any destination that is NOT in Approved\_DNS\_Servers list.

### Testing:

Test 1: pfSense accessible  
Result: SUCCESS

Test 2: nslookup google.com  
Result: FAILURE - "server failure"

Test 3: Browsing  
Result: FAILURE - No connectivity

**Failure Analysis:** Invert match did not resolve the issue. DNS Resolver still unable to function with any form of port 53 blocking active, regardless of rule logic.

**Rollback:** Disabled block rule → Network restored

---

## Attempt 3: Localhost Exception for Internal Operations

**Approach:** Add explicit allow rule for localhost (127.0.0.1) to permit pfSense's internal DNS operations before implementing block rule.

**Theory:** pfSense DNS Resolver might use localhost (127.0.0.1) as source when forwarding queries internally. An exception for localhost could allow internal operations while still blocking unauthorized client queries.

### Implementation:

Added Localhost Allow Rule (positioned first):

Action: Pass  
Interface: LAN  
Protocol: TCP/UDP  
Source: Single host - 127.0.0.1  
Destination: any  
Destination Port: DNS (53)  
Log: Enabled  
Description: Allow pfSense localhost DNS operations

#### Rule Order:

1. Anti-Lockout Rule
2. Allow pfSense localhost DNS operations [NEW]
3. Allow pfSense to forward DNS to Pi-hole
4. Allow clients DNS to approved servers only
5. Block DNS to unauthorized servers (with invert match)
- ...

#### Testing:

Test 1: pfSense accessible  
Result: SUCCESS

Test 2: nslookup google.com  
Result: FAILURE - "server failure"

Test 3: Browsing  
Result: FAILURE - No connectivity

**Failure Analysis:** Localhost exception did not solve the problem. pfSense DNS Resolver's internal traffic does not present itself as source 127.0.0.1 on LAN interface firewall rules. The architectural issue persists.

**Rollback:** Removed localhost rule and disabled block rule

---

### Attempt 4: DNS Resolver Access Lists

**Approach:** Instead of firewall rules, use pfSense DNS Resolver's built-in Access Lists feature to control DNS access at the application layer.

**Theory:** Access Lists operate within DNS Resolver service before queries are processed, potentially avoiding firewall rule conflicts.

## Implementation:

Navigate: Services → DNS Resolver → Access Lists tab

Created Access List:

Name: Management\_Network\_DNS\_Control

Action: Allow

Networks: 192.168.10.0/24

Description: Allow Management network DNS queries through resolver

**Analysis:** pfSense DNS Resolver Access Lists control who can USE the DNS Resolver service, not whether clients can bypass it entirely by querying other servers.

**Finding:** Access Lists do NOT provide enforcement against DNS bypass. They only control access to pfSense's own DNS Resolver, not firewall-level restriction of port 53 traffic to unauthorized destinations.

**Result:** Approach fundamentally incompatible with objective. No enforcement capability provided.

**Cleanup:** Deleted Access List configuration

---

## Attempt 5: NAT Port Redirect (Transparent Proxy)

**Approach:** Use NAT port forward to intercept ALL port 53 traffic and redirect to pfSense (127.0.0.1) regardless of client's intended destination.

**Theory:** NAT processing occurs BEFORE firewall rules, potentially avoiding the architectural conflicts. Clients attempting to query 8.8.8.8 would be transparently redirected to pfSense DNS Resolver.

## Implementation:

Navigate: Firewall → NAT → Port Forward

Created NAT Redirect Rule:

Interface: LAN

Protocol: TCP/UDP

Source: LAN net

Destination: any

Destination Port: DNS (53)

Redirect Target IP: 127.0.0.1

Redirect Target Port: DNS (53)

Description: Force all DNS queries to pfSense resolver

Filter Rule Association: None

## Testing:



Test 1: pfSense accessible

Result: SUCCESS

Test 2: nslookup google.com

Result: FAILURE - "server failure"

Test 3: Browsing

Result: FAILURE - No connectivity

**Failure Analysis:** NAT redirect also breaks DNS resolution. Redirecting all port 53 traffic to localhost creates routing or processing conflicts within pfSense DNS Resolver that prevent successful query completion.

**Rollback:** Disabled NAT redirect rule → Network restored

---

## Failure Analysis

### Root Cause: pfSense DNS Resolver Architecture

After five systematic implementation attempts, the consistent failure pattern reveals a fundamental architectural incompatibility.

#### pfSense DNS Resolver Internal Architecture:

Client Query Flow (Forwarding Mode):

1. Client (192.168.10.4) → pfSense (192.168.10.1:53)
2. pfSense DNS Resolver receives query
3. Internal process forwards to Pi-hole (192.168.10.106:53)
4. Pi-hole queries upstream DNS (1.1.1.1 or 8.8.8.8)
5. Response chain: Upstream → Pi-hole → pfSense → Client

#### Critical Requirements:

- DNS Resolver must listen on "Network Interfaces: All" (localhost + LAN + potentially others)
- Reason: pfSense system itself uses 127.0.0.1 for DNS resolution
- Attempting to restrict to LAN-only produces error: *"The system is configured to use DNS resolver as its DNS server, so localhost or ALL must be selected"*

#### The Incompatibility:

When DNS Resolver operates with "Network Interfaces: All":

- Internal forwarding traffic flows through multiple interfaces and processes
- Firewall rules on LAN interface cannot adequately model these internal flows

- ANY restriction on port 53 (block rules, NAT redirects) interferes with legitimate internal forwarding
- The forwarding mechanism requires traffic patterns that don't cleanly match simple firewall allow/block logic

**Evidence:** Every single enforcement mechanism (5 different approaches) consistently broke DNS resolution while pfSense remained accessible, proving the issue is DNS-specific, not general network connectivity.

## Why Each Attempt Failed

**Attempt 1 (Basic Block):** Block rule prevented pfSense DNS Resolver from completing forwarding operations to Pi-hole

**Attempt 2 (Invert Match):** Same issue - block rule logic still interfered with internal forwarding regardless of invert match

**Attempt 3 (Localhost Exception):** pfSense internal traffic doesn't present as 127.0.0.1 source on LAN interface rules

**Attempt 4 (Access Lists):** Fundamentally wrong tool - controls DNS Resolver access, not firewall-level port 53 enforcement

**Attempt 5 (NAT Redirect):** Redirecting all port 53 to localhost creates routing loops or processing conflicts within DNS Resolver

---

## Alternative Approaches Considered

### Option A: Accept the Vulnerability (Recommended)

**Approach:** Document DNS bypass as a known architectural limitation, proceed with other security projects.

#### Rationale:

- 3+ hours invested with zero success across five different methods
- Fundamental architectural incompatibility confirmed
- DNS tunneling is ONE attack vector among many
- Lab has substantial detection capability for most attack types:
  - Wazuh: Host-based detection across 6 systems
  - Zeek: Network monitoring on Internal and User-LAN segments
  - pfSense: Perimeter and routing logs
  - Pi-hole: DNS logging for compliant clients

**Risk Assessment:** Medium severity. DNS tunneling remains undetected, but:

- Requires attacker to already have access to configure client DNS settings

- Post-compromise technique, not initial access vector
- Other lateral movement and command-and-control techniques are detectable
- Industry-wide challenge (even enterprises struggle with DNS tunneling detection)

**Implementation:** None required. Document limitation and proceed.

**Time Investment:** 0 hours additional

## **Option B: DNS Architecture Redesign (High Risk)**

**Approach:** Disable pfSense DNS Resolver, configure Pi-hole as primary DNS for all clients and pfSense itself, implement firewall blocks.

### **Implementation Steps:**

1. Configure pfSense to use Pi-hole (192.168.10.106) as System DNS
2. Disable DNS Resolver service entirely
3. Modify DHCP to push Pi-hole IP directly to all clients
4. Implement firewall rules blocking all DNS except to Pi-hole
5. Test pfSense services that depend on DNS
6. Validate all network clients can resolve DNS
7. Test DNS tunnel to confirm block functionality

**Rationale:** Eliminates forwarding chain complexity. Simple architecture: Clients → Pi-hole (direct), no pfSense DNS processing.

### **Risks:**

- pfSense services requiring DNS may break (updates, package management, NTP)
- Pi-hole becomes single point of failure (no redundancy)
- Troubleshooting becomes more complex
- Extended outage likely during transition (2-3+ hours)
- Unknown compatibility issues with pfSense internal services

**Time Investment:** 2-3 hours minimum, potentially more if complications arise

**Success Probability:** Moderate (60-70%). Simpler architecture should allow firewall enforcement, but pfSense may have unexpected dependencies on its own DNS Resolver.

## **Option C: Different Firewall/DNS Platform (Out of Scope)**

**Approach:** Replace pfSense with different firewall platform or implement dedicated DNS security appliance.

### **Examples:**

- OPNsense firewall (pfSense fork with different architecture)
- Dedicated DNS filtering appliance
- Custom Linux firewall with dnsmasq or bind

**Rationale:** Different platforms may not have same architectural constraints.

**Assessment:** Out of proportion to problem being solved. Replacing entire firewall infrastructure to address one attack vector is excessive for lab environment.

**Time Investment:** Days to weeks

**Recommendation:** Not viable for current project scope

---

## Conclusions and Recommendations

### Technical Conclusions

1. **pfSense DNS Resolver in forwarding mode is architecturally incompatible with LAN interface port 53 blocking.** Five different enforcement approaches all failed with identical symptom (SERVFAIL), confirming systematic architectural issue rather than configuration error.
2. **The DNS bypass vulnerability from Project 6 cannot be closed using firewall rules or NAT** without major DNS infrastructure redesign.
3. **DNS Resolver's requirement for "Network Interfaces: All"** creates internal traffic flows that cannot be adequately modeled in pfSense firewall rules on the LAN interface.
4. **Allow-only rules provide zero enforcement** because the default allow rule processes after them, permitting bypass traffic anyway.

### Recommendations

#### Immediate Action: Accept the Limitation

Document DNS tunneling as an undetected attack vector in current architecture. The 3+ hour time investment with zero progress across multiple approaches demonstrates diminishing returns.

#### Focus security efforts on attack vectors where detection exists:

- Wazuh host-based detection (file integrity, process monitoring, authentication)
- Zeek network behavior analytics (protocol analysis, connection patterns)
- pfSense perimeter logging (blocked connections, NAT translations)
- Cross-tool correlation in ELK (Project 8)

#### Risk Mitigation Without Enforcement:

- Endpoint security (ensure Wazuh agents on critical systems)

- Network segmentation (limit lateral movement if DNS tunnel established)
- Behavioral analysis (Zeek may detect post-compromise activities even if tunnel invisible)
- Regular security assessments to detect compromised systems through other indicators

### **Future Consideration (Low Priority):**

If DNS tunneling detection becomes critical requirement:

- Evaluate Option B (Pi-hole primary DNS redesign) during scheduled maintenance window
- Consider dedicated DNS security appliance for production environments
- Explore cloud-based DNS filtering services (Cloudflare Gateway, Cisco Umbrella) for enforcement at provider level

## **Project 6a Status**

**Status:** FAILED - Objective not achieved

**Time Invested:** 3+ hours systematic testing

**Approaches Attempted:** 5 distinct methods

**Success Rate:** 0/5 (100% failure rate)

**Architectural Lesson Learned:** Enterprise firewall DNS integration complexity can create scenarios where security enforcement is incompatible with service operation. Sometimes accepting documented risk is more pragmatic than fighting architectural battles.

## **Impact on Overall Lab Security Posture**

### **Positive Security Capabilities (Remain Intact):**

- Host-based detection across 6 critical systems (Wazuh)
- Network behavior monitoring on 2 key segments (Zeek)
- Perimeter security and logging (pfSense firewalls)
- Centralized log aggregation (Wazuh SIEM)
- Compliance framework mapping (PCI DSS, HIPAA, NIST, GDPR)

### **Known Gap (Documented):**

- DNS tunneling attacks: Undetectable with current architecture
- Impact: Post-compromise C2 channel establishment may succeed without detection
- Mitigation: Other detection layers (Wazuh, Zeek) may identify post-compromise activities

**Overall Assessment:** Lab maintains strong detection capability for 80%+ of attack vectors. DNS tunneling is a specific gap, but not a systemic failure of security architecture.

---

## References

### Documentation Resources

1. **pfSense Firewall Rules Guide:** <https://docs.netgate.com/pfsense/en/latest/firewall/>
2. **pfSense DNS Resolver Configuration:**  
<https://docs.netgate.com/pfsense/en/latest/services/dns/resolver.html>
3. **pfSense NAT Documentation:** <https://docs.netgate.com/pfsense/en/latest/nat/>
4. **Project 6 Implementation Guide:** DNS Tunneling Detection & Security Gap Analysis

### Technical Standards

1. **RFC 1035:** Domain Names - Implementation and Specification
2. **NIST SP 800-81-2:** Secure Domain Name System (DNS) Deployment Guide
3. **DNS Security Best Practices:** SANS Institute Guidelines
4. **Firewall Policy Development:** Enterprise Security Architecture Principles

### Related Lab Projects

1. **Project 6:** DNS Tunneling Detection & Security Gap Analysis (Identified the vulnerability)
2. **Project 3:** Pi-hole DNS Filtering Implementation (Current DNS architecture)
3. **Project 1:** pfSense Multi-VLAN Configuration (Firewall baseline)
4. **Project 8** (Upcoming): ELK Stack SIEM Integration (Focus on detectable attack vectors)

---

**Document Version:** 1.0

**Last Updated:** October 1, 2025

**Author:** Prageeth Panicker

**Project Status:** FAILED - Architectural Incompatibility Confirmed

**Next Steps:** Accept limitation, proceed to Project 8 (ELK integration)

---

*This document serves as a comprehensive failure analysis and lessons-learned report for cybersecurity professionals attempting DNS enforcement in pfSense environments. The systematic approach and detailed failure documentation provide valuable insights into firewall DNS architecture limitations that may inform design decisions in production environments.*